

Making a Shape Generator a Scene Builder: Fine-Grained Scene-Level Metric-Semantic Maps

Anonymous Submission: SeMaNa @ IROS 2026

Abstract—Models such as SAM3D generate high-quality object meshes from a single view and mask. We show that such a “shape generator” is a poor “scene builder”. Such models do not lead to globally consistent and precise maps because they do not reason precisely about the locations of objects in the world, their sizes and physical constraints. We argue that although SAM3D as a generator is very promising, it needs to be paired with classical mapping machinery to get accurate maps. We therefore argue for a front-end that segments and tracks objects and a metric back-end that explicitly registers and scales each generated mesh against a pointcloud. The result is a metric-semantic map where each node is a globally posed and semantically labeled object mesh. In ProcTHOR simulation and on a Unitree Go2 quadruped robot with a RealSense camera, we show that (i) registration improves the pose and scale of predicted objects over raw SAM3D predictions (object localization IoU-F1 $0.40 \rightarrow 0.49$; scale error $0.32 \rightarrow 0.27$), (ii) this approach can generate meshes corresponding to object views that the robot never observed before, and (iii) in real-world experiments, our mapping approach is more accurate than state-of-the-art scene-level mapping pipelines such as Clio (Maggio et al., 2024). Finally, we show how an LLM can use the JSON-based metric-semantic map to prescribe semantic navigation waypoints for a robot to execute autonomously.

I. INTRODUCTION

For tasks ranging from search and rescue, industrial automation, and guiding people through complex environments like airports or museums, a majority of robotic systems use metric representations of the environment for mapping and navigation. Humans use more abstract representations for such tasks. Structures such as place cells and grid cells in the brain [1, 2] encode space efficiently [3] and are tuned to sensory and contextual cues [4, 5]. These structures have also been argued to represent semantics for more general kinds of decision making [6]. The hallmark of human representations of space is indeed their semantic richness—often in lieu of metric details. It stands to reason that robots will benefit by combining semantic information with metric information for building effective representations of space.

Existing approaches in the literature lie at two ends of a spectrum. Scene-level maps (e.g., simultaneous localization and mapping (SLAM) over voxels, neural radiance fields (NeRFs) or Gaussian splats (3DGS), and scene-graph systems built upon these representations capture global metric layout and use language-based priors to create metric-semantic maps. These approaches do work well but they are often tailored to specific tasks, e.g., vision-language models for navigation, language-guided neural radiance fields for mapping, etc. However, these methods lack realistic fine-grained geometry.

On the other end of the spectrum, generative models such as SAM3D [7] or SceneGen [8] synthesize high-fidelity object meshes within a localized, bounded scene area. While the latter can leverage multiple images for better object quality, it

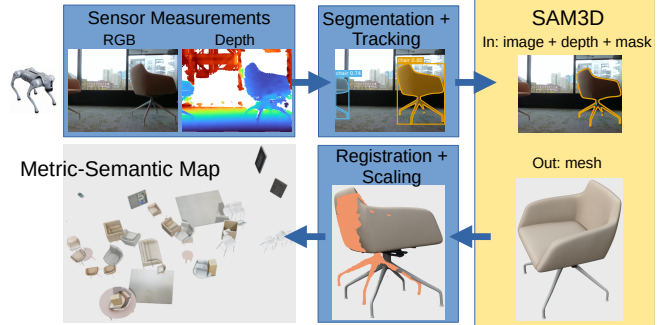


Fig. 1: Our robot collects sensor measurements (RGB-D and odometry) and performs semantic segmentation and tracking over relevant objects in the scene. SAM3D is used to generate the mesh for each object using the best view along its track. Each object mesh is then registered and scaled against the accumulated pointcloud to localize within global scene to produce our metric-semantic map.

is not designed for large-scale SLAM, i.e., it cannot integrate long trajectories to build a representation of a large area. Conversely, multi-view generative models [9, 10] address large-scale scene synthesis by prioritizing global spatial consistency across camera trajectories and multi-view imagery. However, these models typically produce monolithic representations, such as a single continuous mesh or a global radiance field, where objects are fused with the background. The lack of instance-level segmentation in these representations creates a significant gap for robotics applications, which require the ability to isolate and semantically identify individual objects.

We argue that a synergy between classical mapping techniques that can use multi-view observations with generative 3D models is necessary to create fine-grained scene-level metric-semantic maps. Such representations can also readily capture scene-level details, e.g., as a JSON that contains pointers to learned meshes, to enable the use of LLMs for robot autonomy tasks. To this end, the contributions of this work (shown in Fig. 1) are as follows. We demonstrate in simulation (ProcTHOR [11]) and in the real world (Unitree Go2 quadruped robot) that (i) explicit registration against the pointcloud supplies scene-level metric pose and scale that SAM3D cannot predict accurately (Sec. IV-B). (ii) Generative models on the other hand are useful because they can complete unseen geometry (Sec. IV-C). (iii) Our approach outperforms Clio [12] in mapping accuracy with fine-grained semantic objects (Sec. IV-D). We also show that this map representation can be used by an LLM to predict waypoints for autonomous semantic navigation by the robot (Sec. V).

II. RELATED WORK

3D metric-semantic scene graph approaches first construct a geometric representation of the scene using simultaneous localization and mapping (SLAM) on 3D voxels built from signed distance fields [13] or newer techniques like neural radiance fields (NeRFs) [14, 15] and Gaussian splatting (3DGS) [16], augmented with semantic labels [17]. Hydra [18] and Khronos [19] derive object nodes by clustering the global labeled map. ConceptGraphs [20], SuperMap [21], and Clio [12] lift per-frame open-vocabulary masks into 3D and fuse these into per-object pointclouds. Clio also merges its fused primitives into task-relevant objects. Our front-end follows these techniques, however, the difference is that each node of the map is a localized, simulation-ready mesh rather than a labeled point cluster.

Real to sim was studied as “virtualized reality” by Kanade and colleagues [22]. It has again become important because simulation-based robotics requires realistic virtual versions of physical scenes. Recent work reconstructs the scene with 3DGS for a physics simulator [23, 24], builds scenes consisting of objects from a single RGB image [25], or synthesizes scenes from text [26].

Single-image 3D generation has advanced rapidly: TRELIS [27, 28] and SAM3D [7] reconstruct the full object from a single image, RGBD data and/or an object mask. These models typically produce meshes with accurate *shapes*. They are not as accurate in object localization within the scene. Our back-end is agnostic to which method is used to build the mesh. Our experiments in this paper use SAM3D, and while it effectively estimates unobserved geometry and textures, its reliance on learned flow matching [29] for pose prediction—rather than explicit optimization—introduces large errors. We therefore recover meshes using SAM3D and estimate object pose and scale by registration against the sensor data.

III. METHODS

We assume that the robot has an estimate of its pose in $SE(3)$ at each time instant, i.e., there exists a system for localization or odometry, and the robot observes a calibrated RGB-D stream. Our goal is to build a global scene-level representation which consists of “nodes” that are semantically labeled 3D meshes localized accurately within the scene. Our pipeline (Fig. 1) has three stages: a front-end that decides *what objects exist* (Sec. III-A), a single-image-based 3D generator that supplies each node’s *shape* (Sec. III-B), and a metric back-end that decides *where and how big* each object is (Sec. III-C).

A. Front-End

YOLOE [30], an open-vocabulary detector and segmentation model prompted with the scene vocabulary runs on each RGB frame to obtain 2D object masks with labels and confidences. Following the tracking-by-detection paradigm [20], we back-project each detection and associate it across frames into a *semantic object track* k using a cost consisting of mask overlap, appearance, and centroid proximity. The track accumulates a voxel-based pointcloud P_k . Tracks require at least three observations to be considered non-spurious. The best view of this track that is the least-occluded and

approximately centered within the view is used for mesh generation.

B. Generative Node: SAM3D, Shape Only

SAM3D [7], a single-image-based generative model that we use to reconstruct the object mesh from the best view from each track. We pass the full RGB-D frame together with the object mask.¹ SAM3D also predicts the object pose, but it does so by flow matching in normalized camera coordinates. We found that, in practice, this pose is not sufficiently accurate or robust to yield reliable mapping. We discard the predicted translation but use the predicted rotation and scale to initialize an explicit registration procedure.

SAM3D requires about 21 sec. (median) per object on an Nvidia 5090 (18–27 sec). Each object is passed asynchronously to SAM3D and therefore mapping cost scales with the number of objects rather than the length of the trajectory. The rest of the pipeline is fast (on the real hallway scene, detection runs at ~ 12 ms and tracking at ~ 28 ms per frame; estimating the pose and scale of each object takes ~ 1.5 sec per object. A new object node is added to the map about 30 sec after its best view is chosen. The maps in Sec. IV are built offline from recorded data.

C. Metric-Placement Back-End

We resolve the pose (rotation and translation) and a per-axis metric scale of each node via optimization as follows. We back-project each detection mask into the world using our known camera intrinsics and extrinsics, accumulate the fused cloud P_k across the entire track and register the mesh surface to P_k by point-to-point ICP [31]. ICP is used to improve translational and rotational localization. The scale is estimated using the silhouette as follows. We observe that the object mask in the image tends to be a reliable even if the pointcloud is noisy. For each object k , we estimate a per-axis scale s (about the center of the 3D object center) such that the rendered projection of the generated object onto the image plane matches the observed masks across views along the track $\mathcal{V}_k$,

$$s_k^* = \arg \min_s \sum_{v \in \mathcal{V}_k} [1 - \text{IoU}(\pi_v(M_s), m_v)] + \lambda \|s - 1\|^2, \quad (1)$$

where $\pi_v(M_s)$ is the silhouette of the scaled mesh (M_s) in view v , m_v the observed mask, and a small λ pulls weakly-observed axes towards the size estimated by SAM3D (which is $s=1$ here). The number of views used to compute Eq. (1) depends upon the reliability of the estimated object pose. In simulation the camera poses are mutually consistent, so $\mathcal{V}_k$ spans the entire track and the viewpoints jointly pin all three axes. On the real robot, odometry drift makes cross-frame rendering inconsistent, so we set $\mathcal{V}_k$ to be the single best view. In that case Eq. (1) has a closed form that does not require rendering or cross-frame pose estimates. We match the projection of the mesh bounding box in the image plane to the mask’s principal pixel spans at the median depth of the mask $\tilde{d}$ (a span of s_{px} pixels is $s_{\text{px}}\tilde{d}/f$ meters at focal length f). The scale of the object along the axis orthogonal to the image plane is set to correspond to the aspect ratio of the object estimated by SAM3D.

¹We noticed that providing the full frame instead of a crop substantially reduces localization error because it provides more context to the model.

IV. EXPERIMENTAL EVALUATION

In simulation we use ten held-out scenes from ProcTHOR [11] which generates furnished indoor houses. We render RGB-D frames with poses by performing a walk-through of each scene and use ground truth object meshes to evaluate the fidelity of the map directly. For real-robot experiments, we use a Unitree Go2 quadruped in four indoor scenes. The robot uses a RealSense RGB-D camera and has onboard odometry. Ground truth is computed from hand-labeled oriented boxes. We run external baselines such as Clio [12] using the same ground truth and metrics.

A. Evaluation Metrics

We are interested in evaluating both the precision/recall of object detection as well as its accurate pose localization and size/shape in the map. We use two kinds of metrics. Predictions are matched to ground truth by Hungarian assignment on 3D oriented-boxes (IoU); IoU-F1 is the F1 over matched pairs at $\text{IoU} \geq 0.25$, and $\text{recall}@0.5$ repeats the recall at $\text{IoU} \geq 0.5$. On matched pairs, we report centroid (cent), orientation (rot), and scale error (scale.err, the largest per-axis relative extent error), and the Scan2CAD (S2C) [32] accuracy which requires all three to be within 20 cm/20°/20% *simultaneously*—this is the most strict test for a drop-in mesh. As a loose counterpart, we also evaluate to check if top-down centroid is within 1 m and report a label-agnostic centroid-distance F1 (cd-F1) and class-free recall (cf).

B. Shape priors from SAM3D require registration to obtain accurate scene reconstruction

Out of the box, SAM3D returns a full object local pose alongside its mesh, so it is reasonable to ask whether its estimates are good enough to build an accurate map if given global camera pose. We place every generated object mesh in three ways: using SAM3D’s predicted pose and global camera pose (SAM3D), by registration to the fused depth cloud (+ICP), and by additionally fitting scale to the observed masks using Eq. (1) (+scale+ICP). We compare against the bounding boxes generated by the Clio [12] scene-graph methods.

TABLE I: Simulation ablation (ProcTHOR val-10, 10-scene mean), front-end fixed. Best per column in **bold**.

config	IoU-F1 $\uparrow$	rec@.5 $\uparrow$	S2C $\uparrow$	rot° $\downarrow$	scale.err $\downarrow$
Clio	0.41	0.23	0.12	—	0.31
SAM3D	0.40	0.15	0.08	9.7	0.32
+ICP	0.49	0.22	0.09	6.8	0.32
+scale+ICP	0.48	0.23	0.14	6.1	0.27

SAM3D’s native estimate only gets 0.40 IoU-F1; registering (+ICP) improves it to 0.49 and cuts rotation error $9.7 \rightarrow 6.8^\circ$. In other words, SAM3D produces the right shape but places it poorly. Fitting scale cuts the scale error $0.32 \rightarrow 0.27$ and improves Scan2CAD $0.09 \rightarrow 0.14$. The gain is modest in simulation, where SAM3D’s scale is already in distribution but pays off on real robot data in Table II. Against the baseline, our approach shows improvement on all metrics (IoU-F1, Scan2CAD, scale error). The generated map is also visibly cleaner (Fig. 2). The baseline method is competitive only on the loose metrics that detect whether the ground truth object matches the labeled pointcloud.

C. Generation Completes the Unobserved Geometry

To separate shape from placement, for each object o with ground-truth surface G_o and fused observed cloud O , we place the object at an oracle pose and measure the observed coverage c (at $\tau=5$ cm) and the reconstruction error $r(X)$ of its mesh X over the *unobserved* surface $U = \{p \in G_o : d(p, O) > \tau\}$:

$$c = \frac{1}{|G_o|} \sum_{p \in G_o} \mathbf{1}[d(p, O) \leq \tau], \text{ and } r(X) = \frac{1}{|U|} \sum_{p \in U} \min_{q \in X} \|p - q\|. \quad (2)$$

This is a Chamfer error over the unseen region (lower is better). We compare the generated mesh on 212 objects in ProcTHOR val-10 using ground truth masks. Our approach helps complete meshes of partially observed objects in sensor data (Fig. 3). It is common in real robotic systems that the robot will generally not have the ability to observe the entire surface of each object due to occlusions, limited viewpoints etc. In this partial observability regime, generation has an unobserved-surface error that is $2.9\times$ lower than the system that only uses the observed pointcloud of an object (0.081 vs. 0.232 m), and across the full test set, generation reduces the error on 166 out of 212 objects. The gain is largest when only a small part of the object is observed and collapses to a tie once an object is well observed (around 18% unseen).

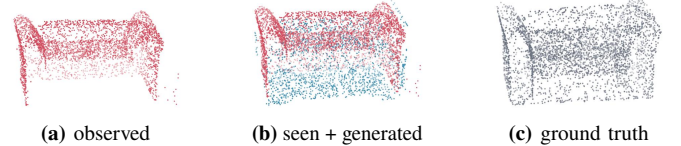
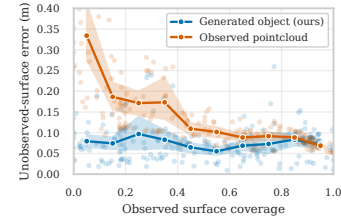


Fig. 3: Generation completes unobserved geometry. *Top:* unobserved-surface error r (Eq. (2)) vs observed coverage c over 212 objects: as observability drops the generated asset’s error r (blue) stays low while the observed pointcloud (orange) degrades sharply. *Bottom:* an armchair at 44% coverage: (a) observed points, (b) observed (red) plus generated mesh (blue) filling the unseen base, (c) ground truth.

D. Object Localization on Real Data

We now evaluate how well the pipeline maps objects in the scene on real hardware. We deploy the full system on data collected from a Go 2 quadruped across four scenes and compare against Clio [12], which reconstructs 3D primitives from per-frame segments and merges them into task-relevant objects with an information-bottleneck objective over CLIP features. We evaluate Clio using the same ground truth and metrics. We also perform ablations on different parts of our pipeline: native SAM3D placement (*SAM3D*), rigid registration (+*ICP*), and our full pipeline (+*scale+ICP*).

On loose open-set recall our mapping solution and Clio tie (Table II; cd-F1 0.63 vs 0.54, class-free 0.75 vs 0.69), i.e. both methods do a decent job at finding objects. On strict placement our map reaches an IoU-F1 0.14–0.19 against Clio’s 0.04. We note that on real data, odometry drift is an inherent issue especially when the robot turns quickly. This

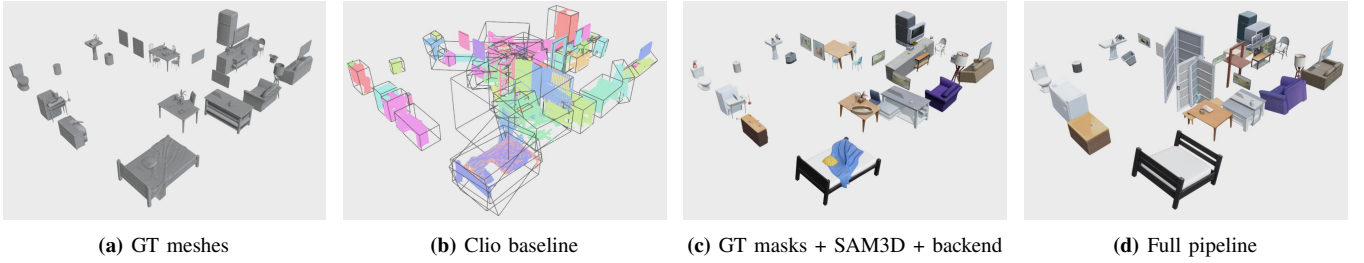


Fig. 2: 3D renderings of the ProcTHOR’s scene 200 via different methodologies. (a) ground truth meshes given by the simulator. (b) Clío’s [12] method has a tendency to over estimate the bounding box of cluttered objects. (c) By providing ground truth masks to the system, we evaluate generation (Sec. III-B) and the back-end (Sec. III-C) in comparison to (d) which uses the full pipeline including the front-end with YOLOE.

TABLE II: Real robot (Go2, 4-scene mean): Clío vs our registration progression (rows below Clío are ours). cent in m; cd-F1/cf use a 1 m centroid match.

config	IoU-F1 $\uparrow$	cent $\downarrow$	scale $\downarrow$	rot $^\circ$ $\downarrow$	cd-F1 $\uparrow$	cf $\uparrow$
Clío (baseline)	0.04	0.32	0.55	—	0.54	0.69
SAM3D	0.17	0.22	0.50	10.8	0.68	0.76
+ICP	0.19	0.22	0.49	16.4	0.68	0.76
+scale+ICP	0.14	0.19	0.35	17.3	0.63	0.75

discrepancy is captured by the rotational error increasing as we rely on the accumulated pointcloud to perform ICP to fix rotation. SAM3D’s initial object orientation prediction is performant in this regime which is grounded to a single image and associated depth. Leveraging this insight, our scale solution utilizes the single-view, pose-free form of Eq. (1) (Sec. III-C) where a single frame’s mask at the observed depth is less corruptible by drift. Although Clío has methods to deal with these inaccuracies in pose, it is more sensitive to these errors nonetheless.

We also notice that Clío tends to over cluster objects due to the ambiguity between multiple instances of a single class. For example, a group of chairs is usually clustered into a single large “chair” by Clío. This happens because repetitive objects will have near similar CLIP embeddings which when in close proximity, the clustering algorithm will associate as one object. In comparison, we see that our method is able to map these repetitive semantic classes better.

V. REAL-WORLD SEMANTIC NAVIGATION

Finally, we close the loop on the hallway scene: the map the robot just built becomes the context an LLM plans over. The experiment goes as follows: (1) we teleoperate the Go2 around the area of interest, building the metric-semantic map; (2) the resulting map is fed as text (JSON) to an LLM (Google’s Gemini) along with a task prompt; and (3) after receiving the response from the LLM, the Go2 autonomously performs waypoint-based path planning and control via Nav2 [33] to move between waypoints. Fig. 4 shows the deployed system diagram and the hallway map.

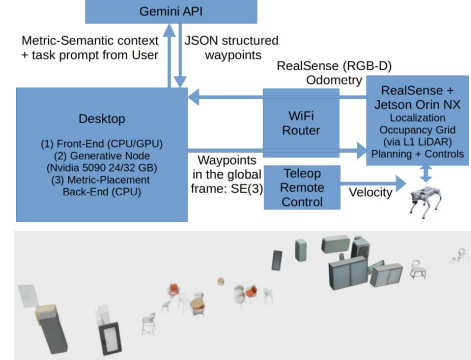


Fig. 4: *Top:* the deployed system. The Go2 streams RGB-D and odometry over WiFi to a desktop running the mapping pipeline; the map and a task prompt go to Gemini, and the returned waypoints are executed onboard (Nav2 on a Jetson Orin NX). *Bottom:* the hallway map handed to the LLM.

Observe that Gemini can discern from our general task of searching for offices that it must extract the locations of doors, seating areas (which are not explicitly defined but rather inferred by Gemini via clusters of chairs), and tables in the hallway scene, in spite of the map being rather noisy. It understands that to find offices, it should navigate along the hallway and check near clusters of furniture which could lead to finding offices.

VI. DISCUSSION

If the most natural way for a human to provide instruction is through language, our robots must be able to interpret these natural language specifications within the context of the semantic and metric information in the environment. A critical procedure to instantiate this paradigm is to build a scene-level representation of the environment that is geometrically and semantically accurate. Out of the box, SAM3D predicts impressively good meshes. But these meshes are not accurate to support robot autonomy, and we therefore need to perform some explicit mapping to build accurate maps at the scene-level. We have demonstrated an approach to build such accurate fine-grained scene-level representations in simulation and with a quadruped Unitree Go2. The key challenges to focus on in future work pertain to (i) improving the robustness and generalization of deep learning-based methods such as YOLO, SAM or SAM3D to real world data which has motion blur from being mounted on a walking platform, illumination and viewpoint changes, as well as nuisances such as glass or shiny surfaces, and (ii) addressing the inevitable computational and communication latency that comes from combining large deep networks into real robotics pipelines.

User: We would like to know how many offices are connected to this hallway. The doors to the offices might not be provided in the data, where might they be? Provide waypoints (x, y) for doors that might not have been seen.

Robot: Here’s a set of waypoints for the robot to follow, incorporating the object locations and a systematic search pattern: Start: (0, 0) - Assuming this is a reasonable origin point. Check Door 1: (-0.5, -12.9). Check Table Area: (-7.8, -6.0). Check Door 2: (-6.3, -10.1). Check Table Area 2: (5.7, -12.8). Check Seating Area 1: (6.0, -12.5). Check Seating Area 2: (13.4, -14.5). Strategy: visit the known doors, traverse the hallway clockwise, and check near clusters of seating and tables, which may sit near entrances to other rooms.

REFERENCES

- [1] E. I. Moser, E. Kropff, and M.-B. Moser, "Place Cells, Grid Cells, and the Brain's Spatial Representation System," *Annual Review of Neuroscience*, vol. 31, no. 1, pp. 69–89, July 2008.
- [2] T. Hafting, M. Fyhn, S. Molden, M.-B. Moser, and E. I. Moser, "Microstructure of a spatial map in the entorhinal cortex," *Nature*, 2005.
- [3] X.-X. Wei, J. Prentice, and V. Balasubramanian, "A principle of economy predicts the functional architecture of grid cells," *eLife*, vol. 4, p. e08362, Sept. 2015.
- [4] Z. Wang, R. W. Di Tullio, S. Rooke, and V. Balasubramanian, "Time Makes Space: Emergence of Place Fields in Networks Encoding Temporally Continuous Sensory Experiences," in *NeurIPS 2024*, 2024.
- [5] Z. Wang, G. *Morris, D. *Derdikman, P. *Chaudhari, and V. *Balasubramanian, "REMI: Reconstructing episodic memory during intrinsic path planning," *arXiv preprint arXiv:2507.02064 (under review at NeurIPS)*, 2025.
- [6] R. Kaplan and K. J. Friston, "Entorhinal transformations in abstract frames of reference," *PLoS biology*, vol. 17, no. 5, p. e3000230, 2019.
- [7] S. D. Team, X. Chen, F.-J. Chu, P. Gleize, K. J. Liang, A. Sax, H. Tang, W. Wang, M. Guo, T. Hardin, X. Li, A. Lin, J. Liu, Z. Ma, A. Sagar, B. Song, X. Wang, J. Yang, B. Zhang, P. Dollár, G. Gkioxari, M. Feiszli, and J. Malik, "Sam 3d: 3dfy anything in images," *arXiv preprint arXiv:2511.16624*, 2025. [Online]. Available: <https://arxiv.org/abs/2511.16624>
- [8] Y. Meng, H. Wu, Y. Zhang, and W. Xie, "Scenegen: Single-image 3d scene generation in one feedforward pass," 2025. [Online]. Available: <https://arxiv.org/abs/2508.15769>
- [9] J. Wang, M. Chen, N. Karaev, A. Vedaldi, C. Rupprecht, and D. Novotny, "Vggt: Visual geometry grounded transformer," 2025. [Online]. Available: <https://arxiv.org/abs/2503.11651>
- [10] H. Lin, S. Chen, J. Liew, D. Y. Chen, Z. Li, G. Shi, J. Feng, and B. Kang, "Depth anything 3: Recovering the visual space from any views," 2025. [Online]. Available: <https://arxiv.org/abs/2511.10647>
- [11] M. Deitke *et al.*, "ProcTHOR: Large-scale embodied AI using procedural generation," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.
- [12] D. Maggio, Y. Chang, N. Hughes, M. Trang, D. Griffith, C. Dougherty, E. Cristofalo, L. Schmid, and L. Carlone, "Clio: Real-time task-driven open-set 3d scene graphs," 2024. [Online]. Available: <https://arxiv.org/abs/2404.13696>
- [13] H. Oleynikova, Z. Taylor, M. Fehr, R. Siegwart, and J. Nieto, "Voxblox: Incremental 3d euclidean signed distance fields for on-board mav planning," in *IEEE/RSJ IROS*, 2017.
- [14] B. Mildenhall, P. P. Srinivasan, M. Tancik, J. T. Barron, R. Ramamoorthi, and R. Ng, "Nerf: Representing scenes as neural radiance fields for view synthesis," in *ECCV*, 2020, pp. 405–421.
- [15] C. D. Hsu and P. Chaudhari, "Active Scout: Multi-target tracking using neural radiance fields in dense urban environments," in *Proc. of International Conference on Intelligent Robots and Systems (IROS)*, 2024.
- [16] B. Kerbl, G. Kopanas, T. Leimkühler, and G. Drettakis, "3d gaussian splatting for real-time radiance field rendering," 2023. [Online]. Available: <https://arxiv.org/abs/2308.04079>
- [17] A. Rosinol, A. Violette, M. Abate, N. Hughes, Y. Chang, J. Shi, A. Gupta, and L. Carlone, "Kimera: from slam to spatial perception with 3d dynamic scene graphs," 2021. [Online]. Available: <https://arxiv.org/abs/2101.06894>
- [18] N. Hughes, Y. Chang, and L. Carlone, "Hydra: A real-time spatial perception system for 3d scene graph construction and optimization," 2022. [Online]. Available: <https://arxiv.org/abs/2201.13360>
- [19] L. Schmid, M. Abate, Y. Chang, and L. Carlone, "Khronos: A unified approach for spatio-temporal metric-semantic slam in dynamic environments," in *RSS*, 2024.
- [20] Q. Gu, A. Kuwajerwala, S. Morin, K. M. Jatavallabhula, B. Sen, A. Agarwal, and et al., "Conceptgraphs: Open-vocabulary 3d scene graphs for perception and planning," 2023. [Online]. Available: <https://arxiv.org/abs/2309.16650>
- [21] S. Zhao, G. Chen, H. Zhu, Z. Li, C. Yao, N. Zantout, S. Kim, W. Wang, J. Zhang, and S. Scherer, "Supermap: A spatio-temporal slam system for visual-language navigation," in *Proceedings of Robotics: Science and Systems (RSS)*, 2026.
- [22] T. Kanade, P. Rander, and P. J. Narayanan, "Virtualized reality: Constructing virtual worlds from real scenes," *IEEE Multimedia*, vol. 4, no. 1, pp. 34 – 47, January 1997.
- [23] X. Li, J. Li, Z. Zhang, R. Zhang, F. Jia, T. Wang, H. Fan, K.-K. Tseng, and R. Wang, "Robogsim: A real2sim2real robotic gaussian splatting simulator," 2024. [Online]. Available: <https://arxiv.org/abs/2411.11839>
- [24] M. Torne, A. Simeonov, Z. Li, A. Chan, T. Chen, A. Gupta, and P. Agrawal, "Reconciling reality through simulation: A real-to-sim-to-real approach for robust manipulation," 2024. [Online]. Available: <https://arxiv.org/abs/2403.03949>
- [25] T. Dai, J. Wong, Y. Jiang, C. Wang, C. Gokmen, R. Zhang, J. Wu, and L. Fei-Fei, "Automated creation of digital cousins for robust policy learning," 2024. [Online]. Available: <https://arxiv.org/abs/2410.07408>
- [26] Y. Lin, J. Humplik, S. H. Huang, L. Hasenclever, F. Romano, S. Saliceti, D. Zheng, J. E. Chen, C. Barros, A. Collister, M. Young, A. Dostmohamed, B. Moran, K. Caluwaerts, M. Giustina, J. Moore, K. Connell, F. Nori, N. Heess, S. Bohez, and A. Byravan, "Proc4gem: Foundation models for physical agency through procedural generation," 2025. [Online]. Available: <https://arxiv.org/abs/2503.08593>
- [27] J. Xiang, Z. Lv, S. Xu, Y. Deng, R. Wang, B. Zhang, D. Chen, X. Tong, and J. Yang, "Structured 3d latents for scalable and versatile 3d generation," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2025, tRELLIS; arXiv:2412.01506, CVPR'25 Spotlight.

- [28] J. Xiang, X. Chen, S. Xu, R. Wang, Z. Lv, Y. Deng, H. Zhu, Y. Dong, H. Zhao, N. J. Yuan, and J. Yang, "Native and compact structured latents for 3d generation," *arXiv preprint arXiv:2512.14692*, 2025, tRELLIS.2.
- [29] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, M. Nickel, and M. Le, "Flow matching for generative modeling," 2023. [Online]. Available: <https://arxiv.org/abs/2210.02747>
- [30] A. Wang, L. Liu, H. Chen, Z. Lin, J. Han, and G. Ding, "Yoloe: Real-time seeing anything," 2025. [Online]. Available: <https://arxiv.org/abs/2503.07465>
- [31] P. Besl and N. D. McKay, "A method for registration of 3-d shapes," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 1992.
- [32] A. Avetisyan, M. Dahnert, A. Dai, M. Savva, A. X. Chang, and M. Nießner, "Scan2CAD: Learning CAD model alignment in RGB-D scans," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019.
- [33] S. Macenski, F. Martin, R. White, and J. Ginés Clavero, "The marathon 2: A navigation system," in *IROS*, 2020.